

Week 4 Report: Testing, Deployment & User Feedback

Team: Fitness Challenge App with Camera-Based Exercise Verification

June 30, 2026

Contents

1	Team & Roles	2
2	Project & Track Statement	2
2.1	Week 4 Goal	2
3	Testing Strategy	2
3.1	Backend Unit & Integration Tests	2
3.2	Test Execution Evidence & Coverage	3
4	CI/CD Pipeline	3
4.1	Continuous Integration	3
4.2	Continuous Deployment	4
5	Environment Setup & Deployment	5
5.1	VM Setup	5
5.2	Docker / Compose Configuration	5
5.3	Deployment Verification	6
6	User Feedback Collection (Industrial Track)	6
6.1	Feedback Methods	6
6.2	Key Findings & Action Items	7
6.3	Next Week Action Plan	8
7	Updated Backlog	8
8	Appendix: Useful Links	10

1 Team & Roles

Team Name: Fitness Challenge App

Repository: <https://github.com/Inno-Fitness/Fitness-Challenge-App>

Name	Email	Responsibility
Ivan	i.lokhmotov@innopolis.university	Product coordination, backend challenges changes, CV improvements view view
Kristina	kr.ushakova@innopolis.university	Design UX for profile feature, challenges, onboarding view view
Elvina	e.belorusova@innopolis.university	CV exercise tracking into frontend view
Daria	d.komzolova@innopolis.university	Backend logic unit tests, challenge backend section view view
Safina	s.mukhamedshina@innopolis.university	Deployment on the server implementation, view
Ekaterina	e.salazkina@innopolis.university	ML model research and development
Anastasia	a.kalashnikova@innopolis.university	Updated UI design adjustments for new features, updated API documentation view
Timur	t.sagdeev@innopolis.university	User feedback, weekly deliverables

2 Project & Track Statement

Project: WowFit Challenges

Track: Industrial

2.1 Week 4 Goal

Ensure product quality through automated testing, set up continuous integration and continuous deployment (CI/CD) pipelines, deploy the application to the provided VM via Docker, and gather initial user and stakeholder feedback on the live product.

3 Testing Strategy

Our testing strategy ensures stability across the backend API, focusing on authentication, challenge business logic, and session tracking. To guarantee reliable and isolated test runs, we utilized in-memory SQLite databases (`test_auth.db` and `test.db`) that provide fast, side-effect-free execution across transactions.

3.1 Backend Unit & Integration Tests

- **Authentication & Security:** Validated user registration, secure password hashing utilizing bcrypt, and the complete JWT token lifecycle (including refresh token

generation and strict unauthorized access prevention).

- **Challenge Mechanics:** Verified end-to-end CRUD operations for challenges, visibility states (private creation vs. public publishing), and complex participation flows such as joining via unique invite codes.
- **Workout Sessions:** Tested the core session submission logic to ensure exercises and daily workout plans automatically close when repetition goals (e.g., `clean_reps` achieving the target goal) are successfully met, triggering accurate streak updates.
- **Profile & Leaderboards:** Ensured accurate sorting of participant leaderboards (prioritizing days completed, streaks, and clean reps) and validated edge cases for the user dashboard, such as successfully filtering out future or unscheduled challenges from the daily plan view.

3.2 Test Execution Evidence & Coverage

- The automated test suite achieved an exceptional overall code coverage of 95% across 1,635 statements[cite: 259, 265].
- The test implementation modules are highly robust, with `test_challenges.py` achieving 100% coverage and `test_auth.py` reaching 99%[cite: 265].
- Core business logic services are thoroughly validated, including database models and schema (100%), `authHandler.py` (97%), `sessionService.py` (92%), and `challengeService.py` (90%)[cite: 265].

The tests reports can be found here: [8] (Test reports folder), where if we follow the provided link, folder "Old tests" corresponds to report on tests conducted at the start of the week, and folder "New tests" corresponds to tests conducted after enhancing the product.

4 CI/CD Pipeline

4.1 Continuous Integration

A CI pipeline is configured using GitHub Actions to run automatically on every push and Pull Request to the `main` branch. You can see the description of each of them on our project's Github Actions page:

<https://github.com/Inno-Fitness/Fitness-Challenge-App/actions>

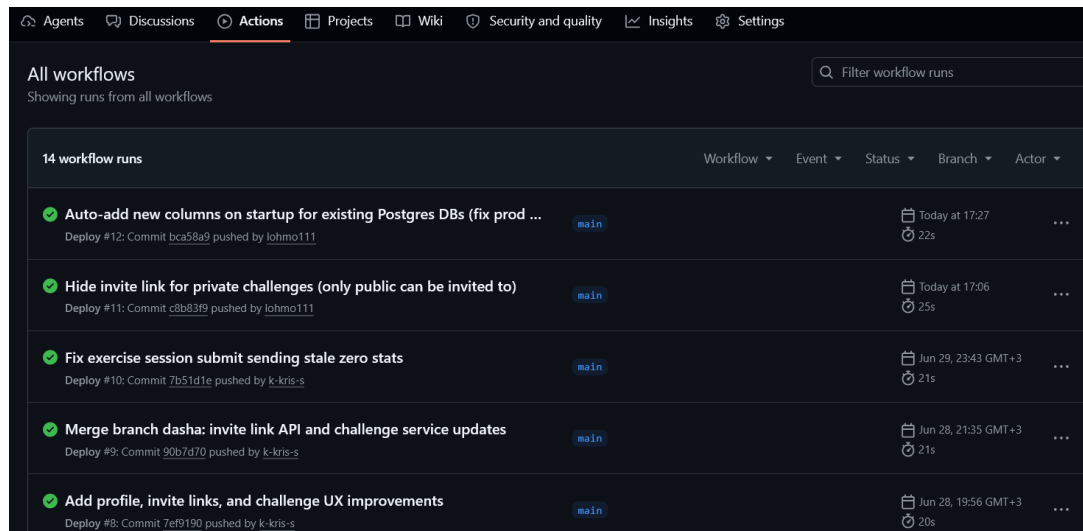


Figure 1: CI workflows on GitHub Actions

4.2 Continuous Deployment

Upon a successful merge to `main`, the CD pipeline triggers an automatic deployment to our VM. This GitHub Actions workflow automates the release process by securely connecting to our production server via SSH. Once authenticated, it navigates to the project directory, pulls the most recent updates from the repository, and executes a detached Docker Compose build to deploy the fresh containers seamlessly. Access the `.github/workflow/deploy.yml` file by the link [8].

Configuration from `deploy.yml`:

```
name: Deploy

on:
  push:
    branches:
      - main

jobs:
  deploy:
    runs-on: ubuntu-latest

    steps:
      - name: Deploy to server
        uses: appleboy/ssh-action@v1.2.0
        with:
          host: ${ secrets.SERVER_HOST }
          username: root
          key: ${ secrets.SERVER_SSH_KEY }
          script: |
            cd /opt/fitness
            git pull origin main
            docker compose up -d --build
```

```
Deploy to server
#27 naming to docker.io/library/fitness-backend:latest done
#27 unpacking to docker.io/library/fitness-backend:latest
#27 unpacking to docker.io/library/fitness-backend:latest 0.4s done
#27 DONE 2.1s
#28 [backend] resolving provenance for metadata file
#28 DONE 0.0s
Image fitness-backend Built
Image fitness-frontend Built
Container fitness-db-1 Recreate
Container fitness-db-1 Recreated
Container fitness-backend-1 Recreate
Container fitness-backend-1 Recreated
Container fitness-frontend-1 Recreate
Container fitness-frontend-1 Recreated
Container fitness-db-1 Starting
Container fitness-db-1 Started
Container fitness-db-1 Waiting
Container fitness-db-1 Healthy
Container fitness-backend-1 Starting
Container fitness-backend-1 Started
Container fitness-frontend-1 Starting
Container fitness-frontend-1 Started
=====
[✓] Successfully executed commands to all hosts.
=====
```

Figure 2: Successful deployment on server logs

5 Environment Setup & Deployment

5.1 VM Setup

The application is deployed on the provided VM at 10.93.27.12. Docker and Docker Compose are installed on the host. Environment variables (e.g., database URLs, secret keys) are securely configured. PostgreSQL was initialized with the necessary schema. However, we decided to use our own server, the access to it can be done by the link: <https://hi-baam.space>

5.2 Docker / Compose Configuration

The system is fully containerized. This configuration file acts as the blueprint for our application's infrastructure. It defines three interconnected services: a robust PostgreSQL database configured with automated health checks and persistent volume storage to prevent data loss, a backend API that depends strictly on the database being fully healthy before it initializes, and the frontend interface. Docker Compose handles the internal networking between these services and exposes the required ports to the external VM environment.

Configuration from docker-compose.yml:

```
services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_USER: user
      POSTGRES_PASSWORD: password
      POSTGRES_DB: postgres
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U user -d postgres"]
      interval: 5s
```

```

    timeout: 5s
    retries: 10
    start_period: 5s
  volumes:
    - pgdata:/var/lib/postgresql/data
  ports:
    - "5432:5432"

  backend:
    build: ./backend
    environment:
      DATABASE_URL: postgresql://user:password@db:5432/postgres
      JWT_SECRET: 9019fd0335fa9d8a97714daa246ebe09
      JWT_ALGORITHM: HS256
      CORS_ORIGINS: "*"
    ports:
      - "127.0.0.1:8001:80"
    depends_on:
      db:
        condition: service_healthy

  frontend:
    build: ./frontend
    ports:
      - "127.0.0.1:3000:80"
    depends_on:
      - backend

  volumes:
    pgdata:

```

5.3 Deployment Verification

Following deployment, smoke tests confirmed that the live application is reachable at <http://10.93.27.12>. User authentication flows function correctly, and challenges successfully persist to the deployed PostgreSQL instance.

6 User Feedback Collection (Industrial Track)

6.1 Feedback Methods

We conducted a live demonstration with the project stakeholder to review the current application, focusing on the updated registration, core challenge workflows, and the computer vision verification mechanics. The goal was to gather actionable requirements for the upcoming development cycles.

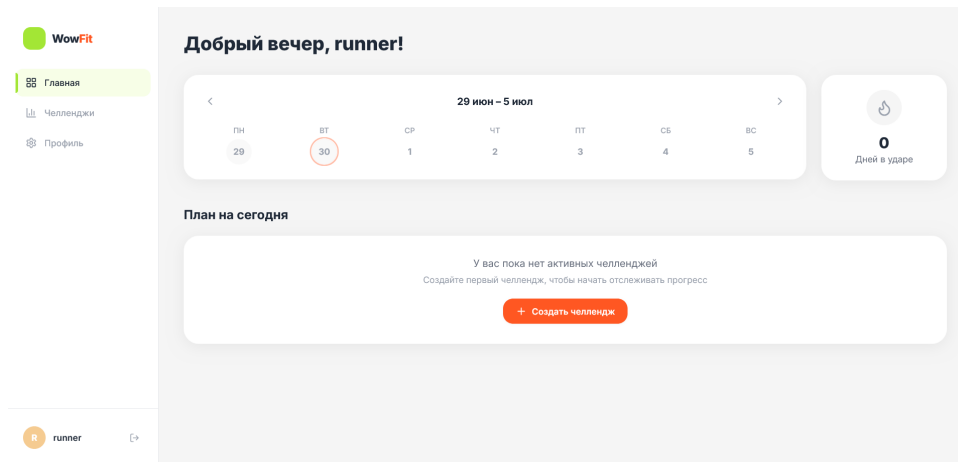


Figure 3: Application deployed

6.2 Key Findings & Action Items

Based on the stakeholder’s review, we have identified the following priority additions and modifications for the project:

- **Authentication & Onboarding:** Consider adding third-party login options (e.g., Google Auth) or simplify the registration process. Enhance the registration page with the WowFit logo and descriptive images.
- **UI & User Engagement:**
 - Make challenge previews more detailed and engaging (consider introducing a virtual currency or reward system).
 - Allow users to click on profiles within the leaderboard to view other participants’ statistics.
 - Make system notifications (e.g., "Link Copied") much more prominent using large modal pop-ups.
 - Add a dedicated section or side panel for educational content, such as technique tutorial videos and articles.
- **Analytics & Tracking:**
 - **Admin Panel:** Create a dashboard for administrators to track user registrations, logins, and referral actions to analyze the target audience.
 - **Health Data:** Integrate Apple Health (or similar services) to display daily step counts in the user’s personal cabinet.
- **Computer Vision Model & Developer Docs:**
 - **User Experience:** Add audio feedback (e.g., a specific sound) when a repetition is successfully counted and improve the model’s flexibility with different camera angles.
 - **Documentation:** Create detailed documentation (e.g., in a README) explaining the current algorithm’s logic, its limitations, and a step-by-step guide on how to add new exercises to the platform in the future.

6.3 Next Week Action Plan

To rapidly address the core findings from this week's stakeholder feedback, next week's sprint will prioritize the following development tasks:

- **Authentication & Graphics:** Simplify the registration process for the user and add the official WowFit branding assets to the registration interface.
- **Engagement UI:** Expand the text-only challenge previews into informative description cards with pictures and build the "Might be interesting for you/News" section on the not used side of the page.
- **Data Integrations:** Research possible ways of solutions and if possible, set up the foundation for the administrative backend to start collecting user activity logs, create the mock integration layer for Apple Health step counts.
- **CV Audio Feedback & Tech Docs:** Embed a responsive audio cue trigger into the frontend application that plays immediately when a repetition is verified, and document the CV pipeline constraints and extensibility steps in the full version of project README.

7 Updated Backlog

Based on the recent stakeholder feedback and our deployment progress, the project backlog has been expanded and reprioritized:

High Priority:

- Develop the foundation for an Administrative Dashboard to monitor user registrations, login activity, and overall engagement metrics.
- Create a dedicated UI section (e.g., a side panel) containing educational content and video tutorials for proper exercise techniques.
- Enable interactive leaderboards, allowing users to click on peers' profiles to view their performance statistics.
- Redesign challenge previews to be more visually engaging and information-dense.

Medium Priority:

- Integrate WowFit corporate branding and descriptive onboarding images into the frontend.
- Add auditory feedback (sound cues) to the exercise verification screen for successful repetitions.
- Improve UI notifications by adding prominent modal pop-ups for actions like "Link Copied."
- Write comprehensive technical documentation detailing the CV model architecture, its limitations, and instructions for adding new exercises.

Long-Term & Research Tasks:

- Integrate third-party health APIs (e.g., Apple Health) to synchronize and display daily step counts within the user dashboard.
- Research and conceptualize a gamification system (e.g., virtual currency or rewards) to increase user retention.
- Optimize the computer vision model to accurately verify exercises from multiple and challenging camera angles.

8 Appendix: Useful Links

- **Repository:** <https://github.com/Inno-Fitness/Fitness-Challenge-App>
- **Github Actions Page:** <https://github.com/Inno-Fitness/Fitness-Challenge-App/actions>
- **CD Workflow:** <https://github.com/Inno-Fitness/Fitness-Challenge-App/blob/main/.github/workflows/deploy.yml>
- **Docker-Compose File:** <https://github.com/Inno-Fitness/Fitness-Challenge-App/blob/main/docker-compose.yml>
- **Live Application:** <https://hi-baam.space>
- **Live Demo:** https://drive.google.com/file/d/1BxhPj0JBBSsED8soDo4PS2TARF_Lr6znc/view?usp=drive_link
- **Test reports folder:** <https://drive.google.com/drive/folders/1gkrWfJXrx9Vbyf294XUguzusp=sharing>